

Everything we know about CRC but afraid to
forget

[illegible]

complex formula is used:

$$\text{CRC}_u M(x); v(x) = v(x) \cdot u(x) \cdot x^{jMj} + M(x) \cdot x^D + u(x) \mod P(x); \quad (1)$$

where polynomial $P(x)$ of degree D and polynomial $u(x)$ of degree less than D are fixed.

The use of the non-zero value of $u(x)$ guarantees that the CRC of a sequence of zeroes is different from zero. That allows detection of insertion of zeroes in the beginning of a message and replacement of botne (

or 64-bit generating polynomials was implemented by the authors in February-March 2007 and was used by a couple of Microsoft products. In 2009, the algorithm was generalized and these limitations were removed.

The fact that the CRC of a message followed by its CRC is a constant value which does not depend on the message, described in section 3.5, is well known and has been widely used in the telecommunication industry for long time.

Proof:

$$\begin{aligned}
 \text{CRC}_u(M; v^0) &= (v^0 \quad u)x^{jMj} + Mx^D + u \mod P = \\
 &= (v^0 \quad u) + (v \quad v) x^{jMj} + Mx^D + u \mod P = \\
 &= (v \quad u) + (v^0 \quad v) x^{jMj} + Mx^D + u \mod P = \\
 &= (v \quad u)x^{jMj} + Mx^D + u + (v^0 \quad v)x^{jMj} \mod P = \\
 &= \text{CRC}_u(M; v) + (v^0 \quad v)x^{jMj} \mod P :
 \end{aligned}$$

3.3 Concatenation of CRCs

If a message $M(x) = M_1(x) x^{jM}$

It is easy to see that

$$\begin{aligned} \text{CRC}_u B^0(x); v(x) - \text{CRC}_u B(x); v(x) &= \\ &= B^0(x) - B(x) x^D \bmod P(x); \end{aligned}$$

so

$$\text{CRC}_u M^0(x); v(x) = \text{CRC}_u M(x); v(x) +$$

where

$$= \text{CRC}_u B^0(x); v(x) - \text{CRC}_u B(x); v(x) x^{iCj} \bmod P(x);$$

3.5 Storing CRC value after the message

Often $Q(x) = \text{CRC}_u M(x); v(x)$ is padded with zero bits until the nearest byte or word boundary and is transmitted as a sequence of W bits ($W \geq D$) right after the message $M(x)$. (This is why the concatenation of $M(x)$ and $Q(x)$ is called

[Has01]:

$$\begin{aligned}
 \text{CRC}_u \uparrow Q(x) x^{W-D}; \text{CRC } M(x); v(x) &= \\
 = \text{CRC}_u \uparrow Q(x) x^{W-D}; Q(x) &= \\
 = Q(x) u(x) x^W + \uparrow Q(x) x^{W-D} x^D + u(x) \bmod P(x) &= x
 \end{aligned}$$

When $D \leq W$,

$$\text{CRC}_0(M(x); v(x)) = v(x) x^W + M(x) x^D$$

```

1  Crc CrcByte(Byte value) f
2      r                               MIByXpowD[value];
3  g
4  Crc CrcByteByByte(Byte data, i t , Crc v, Crc ) f

```

where

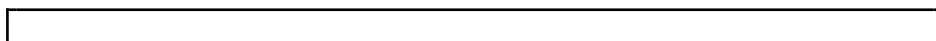
$$F\ C_k(x); m_{k+N}(x) = C_k(x)x^W + m_{k+N}(x)x^D \mod P;$$

$$G\ m_k(x) = v(x) - u(x) x^{NW} + \alpha x x x x \mod m_{k+}$$

x

Consequently,

MuByXpowD $v(x) = v(x) x^D$ m39.969r426 7f 49626 626 Tf 9.9 Td [()]TJ/F8 9.978 Tf 5.186 0 Td [(



```
1 // Processes N stripes of StripeWidth words each
2 // word by word, in an interleaved manner.
```

The drawbacks of this approach are obvious: it does not work well with small inputs { the cost of CRC concatenation becomes a bottleneck, { and it may be susceptible to false cache collisions caused by cache line aliasing.

If the cost of CRC concatenation was not a problem, cache pressure could be mitigated with the use of very narrow stripes. The code in question, lines 33-37 of listing 5 which combine CRCs of individual stripes, iteratively computes

$$\text{crc}_0(x) = \text{crc}_k(x) + \text{crc}_0 \cdot x^{8S} \bmod P(x)$$

for $k = 1; \dots; N - 1$ where N and S

Input message $M(x)$ may be represented as

$$M(x) =$$

mentally:

$$V_{k+1,n}$$

There is more efficient and elegant solution. Assume that $M(x)$ is followed

representation of $v(x)$, viewed as a W -normalized representation, is equal to $v(x) x^{W-D}$.

```

1  Crc CrcInterleavedWordByWord(
2      Word data, int blocks, Crc v, Crc u) f
3      Crc crc[N+1] = f0g;
4      crc[0] = v ^ u;
5      for (int i = 0; i < N (blocks - 1); i += N) f
6          Word buffer[N];
7          // Load next N words and move overflow
8          // bits into "next" word.
9          for (int n = 0; n < N; ++n) f
10             buffer[N] = crc[n] ^ data[i + n];

12             crc[n+1] ^= crc[n] >> (sizeof(Word) - 8);
13             crc[n] = 0;
14             g
15             // Compute interleaved word by

```


functions was better but still not as good as hand-written assembler versions,

Table 1: CRC performance, AMD64 platform

Method	Slicing ¹	Multiword ²	Improvement
CRC-32	2:08 ³	1:16 ^{4;5}	1.79

~~CRC-64~~

1K

404K

Table 4: Compiler comparison: Multiword/4 64-bit CRC

Input size	64	256	1K	4K	16K	64K	256K	1M
GCC/C++								

Table 6: CRC-32 performance

Input size	64	256	1K	4K	16K	64K	256K	1M
Sarwate	6.61	6.62	6.70	6.68	6.67	6.66	6.67	6.75
Black	5.44	5.46	5.47	5.48	5.47	5.46	5.47	5.53
Slicing	2.15	2.10	2.09	2.09	2.08	2.08	2.08	2.10
Blockword/3	2.27	2.14	2.15	2.13	2.13	1.55	1.39	1.31
Multiword/4	1.75	1.29	1.18	1.16	1.17	1.18	1.18	1.18

Number of CPU cycles per byte. 32-bit CRC (CRC-32C polynomial), 64-bit platform, 64-bit tables, 64-bit reads (except Sarwate). Microsoft CL 15.00.30729 compiler. Warm data, warm tables.

\Sarwate" implements the algorithm described in section 4.6.

\Black" implements the algorithm described in section 4.8.

\Slicing" implements the algorithm described in section 4.9.

\Blockword/3" implements the algorithm described in section 4.10 with 3 stripes of 15,376 bytes each.

\Multiword/4" implements the algorithm described in section 4.11 processing 4 data streams in parallel in interleaved manner.

Table 7: CRC-64 performance

Input size	64	256	1K	4K	16K	64K	256K	1M
Sarwate	6.61	6.62	6.70	6.68	6.67	6.65		

Table 8: CRC-128 performance: Slicing CRC

Input size	64	256	1K	4K	16K	64K	256K	1M
CL/SSE2	4.02	3.81	4.01	4.05	4.13	4.18	4.20	4.24
ICL/SSE2	3.40	3.24	3.57	3.59	3.68	3.72	3.75	3.81
GCC/UINT	3.45	3.24	3.36	3.48	3.61	3.64	3.67	3.72
GCC/SSE2	2.67	2.48	2.63	2.79	2.97	2.99	2.99	3.03

Table 9: CRC-128 performance: Multiword CRC

Input size	64	256	1K	4K	16K	64K	256K	1M
GCC/UINT/3	3.83	3.02	3.04	3.01	3.00	2.98	2.98	3.00
CL/SSE2/5	4.08	2.56						